

Homework #2

Instructors: Tanya Goyal

Submission by (netIDs): **kl2235**

Logistics: Read all the instructions carefully before you start working on the assignment, and before you make a submission.

- This is an additional section, meant only for students enrolled in a graduate version of the course, **CS 5740**. If you are enrolled in CS 4740, DO NOT complete this section.
- All CS 5750 students need to complete this assignment **individually**. This assignment is to be done in addition to the regular HW2 assignment.
- Please refer to the course policies on the course webpage to understand how this extra CS5740 component is graded.
- Please typeset your assignment in L^AT_EX. Please include your netID at the top of this page. (Search for "FILL YOUR NETID" to find the appropriate place to enter your netIDs).
- You will submit this assignment on gradescope. We have created a new course on gradescope and added all CS5750 students in there. Please let the instructors know if you have not been added.
- The University Academic Code of Conduct will be strictly enforced.

Section A: Character-level Language Modeling

(S/U points)

In HW2, we saw how “many-to-many” recurrent nets (compared to feed-forward nets), can be effective at identifying and classifying named-entities. In this portion of the assignment, we’ll consider a slightly different and more challenging task of predicting the next *character* (not token) using the given context—you can reason why this a more challenging task.

Recurrent nets have been shown to be quite effective at character-level (language) modeling of English text, source code (e.g., C, L^AT_EX), and others [Graves(2013), Karpathy et al.(2015)Karpathy, Johnson, and Fei-Fei, Madsen(2019)]. For this component, you will be reading Andrej Karpathy’s excellent essay on “[t]he unreasonable effectiveness of [r]ecurrent [n]eural [n]etworks”: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>.¹ A note about long short-term memory nets: they are a special-type of recurrent nets; to complete this assignment, you will not need knowledge of their inner workings.

Once you’ve read the blog post, answer the following questions.

(1) Recurrent nets are quite good at handling sequential data. However, in several applications, data is *truly*² non-sequential; if a single-layered vanilla recurrent net were used to model non-sequential data, how do you expect the hidden state to evolve over time? Is using a recurrent net for such a use-case simply a waste of compute?

(Solution) In a vanilla RNN, the hidden state h_t is designed to carry a memory of previous inputs. If the data is truly non-sequential (independent), the information from h_{t-1} becomes irrelevant noise rather than a helpful context. In that case, RNN will not learn meaningful patterns, and it will be affected by these noise when dealing with current data. I think it is a waste of compute, because RNN must be computed sequentially. In that case, if we don’t have useful information in the previous data, RNN waste it time on finding information on previous data, which is a waste a computation.

¹Code (in Torch 7) to train character-level recurrent nets: <https://github.com/karpathy/char-rnn>.

²Jumbling up the order of inputs doesn’t necessarily change the output; for example, consider the task of classifying if LeBron James will make a three-pointer based on his mood and wind speed.

(2) It is often the case that bidirectional layers³ (one from left-to-right, and other from right-to-left) are used to boost the predictive power of recurrent nets. Explain if using a bidirectional recurrent net is a good choice for the task of character-level language modeling (predicting the next character, given the preceding characters).

(Solution) I don't think bidirectional layers are a good choice to predict, because in the prediction of language model, the model can not get the information from future context and it can only get the information from previous context. If we use bidirectional layers, it will like a "cheat" for model to learn the next token.

(3) For the task of character-level language modeling using a vocabulary V of characters v_i s, we computed the probability of character c_t at timestep t being v_j , given the characters at previous timesteps $c_{<t}$ (a.k.a., context), as:

$$P(c_t = v_j | c_{<t}) = \frac{\exp(y_j/0.1)}{\sum_{k=1}^{|V|} \exp(y_k/0.1)}, \quad (0.1)$$

where y_j is the unnormalized logit associated with v_j , and c_t is chosen as the v_j that maximizes $P(c_t = v_j | c_{<t})$. Once c_t is chosen, we continue the process to choose c_{t+1} using the $c_{<t+1}$ as the context, and so on.⁴

Our sampling strategy resulted in an auto-completion of the text "I am" as "I am *the same user that the same user that the same*" (continuation indicated in *italics*). One can easily see that the generation is bad, in that, the network seems to repeat itself after the token "that"—modify (0.2) to enable more diversity in the generated samples.

(Solution)

$$P(c_t = v_j | c_{<t}) = \frac{\exp(y_j/0.5)}{\sum_{k=1}^{|V|} \exp(y_k/0.5)}, \quad (0.2)$$

Because increasing the temperature parameter from 0.1 to 0.5, the Softmax function becomes less 'sharp', which could reduce the model's tendency to repeatedly pick only the character with the highest logit. In that case, we can break the infinite loop and enable more diverse text generation.

(4) A character-level recurrent net trained on raw HTML source code generated the following: `<ul> <li>item-1</li> <li>item-2</li> </ol>`; note the mismatched `<ul>` (unordered list) opening tag and `</ol>` (ordered list) closing tag. This effect seemed to be pronounced when the number of `<li>item-*` intermediates increased in the output generation.

Explain why such behavior is expected from a recurrent net. What additional information (other than the hidden state and the input) if present at the final timestep would avoid prediction of a mismatched `</ol>`?

(Solution) The reason for that is the hidden state for RNN is merged with new inputs at every time step during the forward propagation. When a large number of intermediate characters are generated, the information from the initial tag will do a lot of matrix multiplications and non-linear activations. In that case, its signal continually decays, which is also the gradient vanishing problem. We could add skip connection to avoid this problem, which will add the initial information to the last hidden layer directly, or use a cell state or gating to preserve long-range information like LSTM.

(5) In an attempt to write poetic essays, we trained a character-level recurrent net on English poems. However, the trained model seems to consistently add a newline character after some fixed length; while this is expected since the line length for poems is often fixed, it is not desired.

To fix this issue, we wish to "post-edit" the learned weights such that the newline character doesn't get appended unnecessarily. Can you think of a way to identify neurons responsible for newline behavior?

Hint: Think how to assess the contributions of individual neurons by visualizing activations.

(Solution) We can feed a large corpus of text into the RNN and monitor the activation values of each hidden unit, and find the specific neurons whose activation levels consistently increase as the line length grows and peak exactly before a newline character is generated. Once the neurons are identified, we can manually decrease or zero out the weights connecting these specific neurons to the output logit for the newline character. By turning off these neurons or adding a negative bias, we can prevent the model from triggering a line break based on fixed length rather than semantic content.

³See: https://d2l.ai/_images/birnn.svg for reference.

⁴This is what is often referred to in the literature as auto-regressive (language) modeling.

References

- [Graves(2013)] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013. URL <https://doi.org/10.48550/arXiv.1308.0850>.
- [Karpathy et al.(2015)] Karpathy, Johnson, and Fei-Fei] Andrej Karpathy, Justin Johnson, and Li Fei-Fei. Visualizing and Understanding Recurrent Networks. *arXiv preprint arXiv:1506.02078*, 2015. URL <https://doi.org/10.48550/arXiv.1506.02078>.
- [Madsen(2019)] Andreas Madsen. Visualizing memorization in rnns. *Distill*, 2019. doi: 10.23915/distill.00016. URL <https://distill.pub/2019/memorization-in-rnns>.